Escuela Politécnica Nacional

Facultad de Ingeniería de Sistemas

Construcción y Evolución del Software – GR2SW

Taller Práctico: Desarmando Gigantes - Ingeniería Inversa en la Práctica

Grupo 02

Integrantes:

- Caicedo Marín Erick Sebastián
- Cuenca Cuenca Josué Sebastián
- Rueda Rueda Ruth Fernanda

Fecha de Entrega: 14 de abril de 2026

Docente: MSc. Adrián Eguez

2026-A

Tabla de contenido

1. Objetivos 1
2. Desarrollo 1
2.1. Fase 1 1
2.2. Fase 2: Investigación Asistida por IA - "Antes, Ahora y Mañana" 2
2.3. Fase 3: El Caso de Estudio - "Hacking Hipotético" 6
2.4. Fase 4: Generación de Imagen para la Exposición 9

1. Objetivos

- Comprender los conceptos fundamentales de la ingeniería inversa y su evolución histórica y tecnológica.
- Analizar e hipotetizar el funcionamiento de una plataforma de software moderna de alto impacto.
- Utilizar la Inteligencia Artificial como herramienta de apoyo para investigar y generar contenido.

2. Desarrollo

2.1. Fase 1

Ingeniería Inversa

La ingeniería inversa, es como "echarse para atrás" en el ciclo de vida del software porque consiste en partir desde el programa ya hecho (a veces sin documentación o con código difícil de entender) y retroceder para descubrir cómo fue construido, cómo funciona y qué lógica tiene. En lugar de diseñar primero y luego programar, se realiza lo contrario: se analiza el resultado final para reconstruir su diseño, estructura y comportamiento.

Mantenimiento de Software

El mantenimiento de software consiste en modificar un sistema después de que ya está en uso, y puede ser de varios tipos:

- correctivo (arreglar errores),
- adaptativo (ajustarlo a nuevos entornos),
- perfectivo (mejorar rendimiento o funcionalidades)
- preventivo (evitar problemas futuros).

La ingeniería inversa es clave aquí porque muchas veces el software es antiguo o no está bien documentado, entonces primero se necesita entender cómo funciona internamente antes de realizar cambios.

Evolución del Software

La evolución del software se refiere a cómo un sistema se adapta y mejora con el tiempo para seguir siendo útil. La ingeniería inversa ayuda a evitar que un sistema se vuelva obsoleto porque permite comprender sistemas antiguos o complejos, facilitando su actualización, migración o integración con nuevas tecnologías. Sin este proceso, sería muy difícil modernizar software legacy, ya que nadie sabría exactamente cómo funciona.

2.2.Fase 2: Investigación Asistida por IA - "Antes, Ahora y Mañana"

Años 90

La ingeniería inversa de software surge como una práctica necesaria cuando los sistemas ya existentes deben ser comprendidos sin contar con suficiente documentación. Históricamente, en los años 60 y 70, los desarrolladores analizaban programas de manera manual, revisando listados impresos y tratando de deducir su lógica. En los años 80, con el crecimiento del software empresarial, se empezaron a formalizar técnicas de análisis, especialmente en lenguajes como COBOL y FORTRAN. Sin embargo, fue en los años 90 cuando la ingeniería inversa tomó mayor relevancia, debido a la gran cantidad de sistemas legacy que necesitaban mantenimiento, migración o corrección sin acceso al diseño original. En esta década, la ingeniería inversa se enfocaba principalmente en el análisis de binarios y código ensamblador **por**que muchas veces no se disponía del código fuente. Las herramientas más utilizadas eran los desensambladores, como IDA Pro, que permitían traducir el código máquina a instrucciones en ensamblador para entender el flujo del programa. También eran fundamentales los depuradores como GDB y SoftICE, que permitían ejecutar el programa paso a paso, inspeccionar registros y analizar el comportamiento en tiempo real. Además, se usaban editores hexadecimales para examinar y modificar directamente los archivos binarios. Las técnicas empleadas en los años 90 eran principalmente de bajo nivel y requerían mucha experiencia. Entre ellas destacaban el análisis estático, que consistía en estudiar el código sin ejecutarlo; el análisis dinámico, que implicaba observar el programa en ejecución; el tracing o seguimiento de instrucciones y la reconstrucción lógica, donde el ingeniero infería estructuras de alto nivel como bucles o condiciones a partir del ensamblador.

Cloud Computing, Microservicios y Ofuscación de Código Moderna

En la era del Cloud Computing, la ingeniería inversa ha evolucionado de analizar programas individuales a estudiar ecosistemas completos en la nube. Antes se trabajaba con un solo ejecutable; ahora los sistemas están distribuidos en múltiples servicios desplegados en infraestructuras dinámicas. Esto ha obligado a cambiar el enfoque hacia el análisis de tráfico, APIs y comportamiento del sistema en tiempo real. Herramientas como Wireshark permiten inspeccionar comunicaciones, mientras que plataformas de observabilidad como Jaeger ayudan a rastrear cómo fluye una solicitud entre distintos servicios. En este contexto, la ingeniería inversa se basa más en entender interacciones que en leer código.

Con la adopción de arquitecturas de microservicios, el desafío se ha vuelto aún más complejo porque la lógica del sistema está fragmentada en múltiples componentes independientes. Esto significa que no existe una única fuente de verdad, sino que el comportamiento global emerge de la comunicación entre servicios. Para enfrentar esto, se utilizan herramientas como Kubernetes y Docker, que permiten inspeccionar contenedores y despliegues. Además, se aplican técnicas como el mapeo de servicios, el análisis de contratos de APIs y el tracing distribuido. En lugar de reconstruir código, el objetivo ahora es reconstruir el comportamiento del sistema completo.

Por otro lado, la ofuscación moderna de código ha elevado significativamente la dificultad de la ingeniería inversa. A diferencia de los años 90, donde el ensamblador era relativamente interpretable, hoy el software puede incluir técnicas avanzadas como cifrado de código, control de flujo ofuscado y mecanismos anti-debugging. Para contrarrestar esto, han surgido herramientas más sofisticadas como Ghidra, IDA Pro y Frida, que permiten analizar y modificar el comportamiento del software en tiempo de ejecución. Las técnicas modernas incluyen instrumentación dinámica, sandboxing y emulación, lo que permite estudiar el programa incluso cuando intenta ocultar su lógica.

En conjunto, se ha pasado de un enfoque centrado en el código y el bajo nivel a uno orientado al análisis de sistemas complejos, distribuidos y protegidos. Hoy, el reto no es solo entender cómo funciona un programa, sino cómo interactúan múltiples componentes en entornos dinámicos y altamente seguros.

Inteligencia Artificial para automatizar la ingeniería inversa de software

La Inteligencia Artificial (IA) introduce avances significativos en la automatización de la ingeniería inversa de software, especialmente en la descompilación asistida. A diferencia de los decompiladores tradicionales que generaban pseudocódigo difícil de interpretar, hoy herramientas como Ghidra, IDA Pro y Binary Ninja integran modelos de IA capaces de reconstruir código más comprensible, infiriendo nombres de variables, estructuras de control y la intención del programa. No solo se traduce el código, sino que se entiende su propósito.

Otro avance importante es el uso de IA en el análisis automático de vulnerabilidades. Actualmente, herramientas como CodeQL y Snyk permiten identificar fallos de seguridad tanto en código fuente como en binarios, detectando patrones asociados a vulnerabilidades como desbordamientos de memoria o malas prácticas de programación. Estas soluciones combinan análisis estático, dinámico y aprendizaje automático, lo que permite no solo encontrar errores, sino también priorizarlos según su impacto, facilitando la toma de decisiones en entornos de desarrollo y mantenimiento.

En el contexto de software protegido, la IA también ha mejorado los procesos de deofuscación automática. Dado que muchos programas modernos utilizan técnicas avanzadas de ofuscación para ocultar su lógica, herramientas como Frida y Radare2 se apoyan en enfoques inteligentes para identificar patrones, simplificar el flujo de control y recuperar información oculta durante la ejecución. Esto permite analizar software que anteriormente era extremadamente difícil de interpretar.

Además, la IA ha revolucionado el análisis de malware, automatizando la clasificación, detección y comprensión del comportamiento de software malicioso. Plataformas como Cuckoo Sandbox y VirusTotal utilizan modelos de aprendizaje automático para analizar grandes volúmenes de muestras, identificar variantes y generar reportes automáticos. Esto ha permitido escalar el análisis de amenazas de manera eficiente y detectar comportamientos sospechosos incluso en malware desconocido.

Ingeniería Inversa

Evolución de herramientas y técnicas

Antes

- Enfoque en binarios individuales
- Análisis manual y de bajo nivel (ensamblador)
- **Herramientas:** IDA Pro, GDB y SoftICE
- **Técnicas:** Desensamblado y Debugging paso a paso

Ahora

- Enfoque en sistemas distribuidos
- Análisis de: APIs, contenedores, tráfico de red
- **Herramientas modernas**: Wireshark, Kubernetes y Jaeger
- **Técnicas:** Observabilidad, Reverse engineering de APIs y Análisis dinámico en cloud

Wireshark

Novedades/IA

- Descompilación inteligente (código más legible)
- Detección automática de vulnerabilidades
- Deobfuscación asistida por IA
- Análisis de malware automatizado
- **Herramientas:** Ghidra, Binary Ninja y CodeQL

La ingeniería inversa evolucionó de procesos manuales a soluciones inteligentes con IA, facilitando el análisis de software complejo.

Grupo 2: Erick Caicedo, Josué Cuenca, Fernanda Rueda

FUENTES

OpenAI. (2026). Resumen ejecutivo sobre la evolución de la ingeniería inversa (Antes, Ahora y Mañana). ChatGPT. https://chatgpt.com/share/69d91505-e0c0-832d-a51e-ce9547dde72a

2.3. Fase 3: El Caso de Estudio - "Hacking Hipotético"

OpenClaw: Sistema de "Task Decomposition" (descomposición de tareas)

La característica de descomposición de tareas (Task Decomposition) en OpenClaw permite que el sistema tome un objetivo complejo y lo divida automáticamente en subtareas más pequeñas, organizadas y ejecutables. Para comprender esta funcionalidad sin acceso al código fuente original, es necesario aplicar técnicas de ingeniería inversa que permitan inferir su comportamiento interno a partir de la observación externa.

En este contexto, se pueden considerar tres enfoques principales: análisis de caja negra y análisis de tráfico de red (API Sniffing).

Análisis de Caja Negra (Black Box)

El enfoque de caja negra consiste en interactuar directamente con el sistema proporcionando distintas entradas y observando sus salidas, sin conocer su implementación interna. Mediante esta técnica se podría:

- Proporcionar diferentes tipos de tareas complejas (ej. planificación, automatización, redacción).
- Analizar cómo el sistema descompone cada tarea en subtareas.
- Observar el orden de ejecución de los pasos generados.
- Identificar si el sistema corrige errores o ajusta el plan dinámicamente.

Este método permite inferir:

- La existencia de un modelo de planificación interna.
- Posibles estrategias como razonamiento paso a paso (Chain-of-Thought).
- Comportamientos iterativos o adaptativos.

Se considera la técnica más importante porque revela directamente el comportamiento funcional del sistema.

Análisis de Tráfico de Red (API Sniffing)

El análisis de tráfico de red implica interceptar y examinar las comunicaciones entre la aplicación cliente y los servidores del sistema. A través de esta técnica se puede:

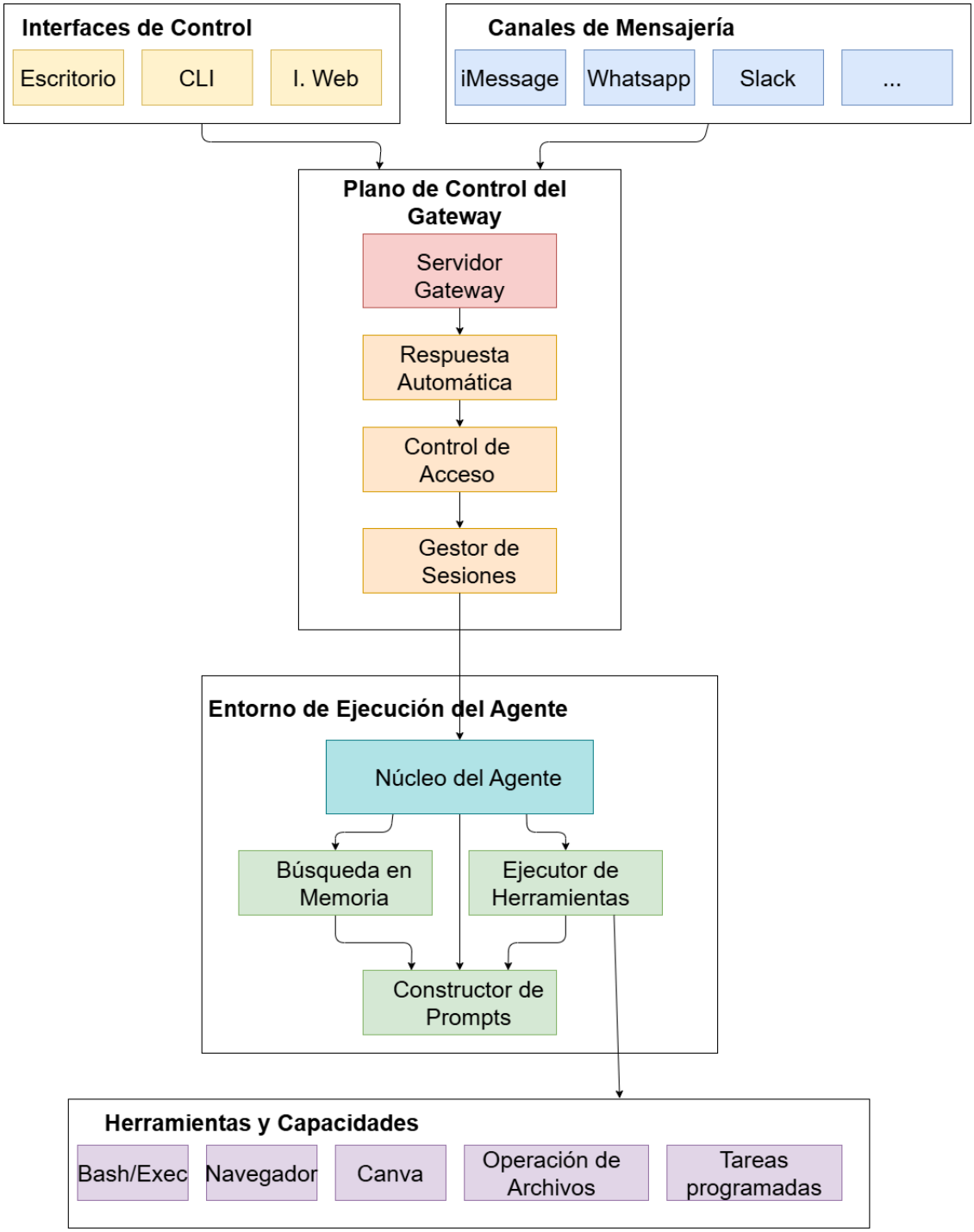
- Identificar las solicitudes (requests) enviadas al backend.
- Analizar las respuestas estructuradas (por ejemplo, en formato JSON).
- Detectar si las tareas se envían y reciben como listas de subtareas.
- Observar si existen múltiples llamadas que indiquen ejecución paso a paso.

Esto permite descubrir:

- La estructura interna de los datos utilizados para representar tareas.
- Si la lógica de descomposición ocurre en el servidor o en el cliente.
- La existencia de flujos de procesamiento o pipelines.

Es una técnica complementaria clave para entender cómo se organiza la información internamente.

ARQUITECTURA



2.4. Fase 4: Generación de Imagen para la Exposición

Prompt: A futuristic, cinematic 3D render without text, visualizing a complex AI integration and reverse engineering process, based on the metaphor of 'OpenClaw' as an orchestral conductor. A central, ethereal conductor with glowing, internal circuitry for arms, made of a complex, transparent material, stands before a vast orchestra of diverse, abstract AI agent entities, each glowing with different colored data-streams (blues, greens, purples). The conductor is meticulously connecting these agents with threads of luminous, geometric, woven digital light, symbolizing the cohesive linking framework. At the conductor's feet, the 'reverse engineering' process is occurring: a portion of the ground is cracked open, revealing a detailed, exploded diagram of the conductor's intricate internal mechanism itself—complex cogs, hidden processor cores, memory buses, and data flow paths, exposed and analyzed. This revealed component, the heart of the conductor, is illuminated by a focused beam of golden light, symbolizing the discovered 'Orchestration and State Management core'. Microscopic, futuristic drones are actively scanning the exposed mechanisms with lasers and sensors, capturing the blueprint. The entire scene is set in a dark, infinite cyber-void with soft, atmospheric lighting, detailed wires, particle effects, and a high contrast, cyberpunk color palette. Minimalist and impactful, focusing solely on the visual story.